

BMAD Method vs Claude Team Skills

Comprehensive Feature Comparison (Updated)

35 features across 7 categories. 5 gaps fixed since initial comparison.

CTS Gaps Remaining	BMAD Gaps	Both Have	CTS Gaps Fixed	Total Compared
8	14	8	5	35

● Has feature ● Partial ● Missing ★ Fixed since initial comparison

NPM installer	npx bmad-method install, interactive prompts	Manual: copy tar.gz into project	CTS gap
Multi-IDE support	10+ IDEs: Claude Code, Cursor, Copilot, Codex, Gemini, etc.	Claude Code only (works in claude.ai with manual paste)	CTS gap
Agent-as-Code (YAML)	YAML source → compiled .md with persona/identity/style	Markdown SKILL.md files, no compilation	CTS gap
Skills-based architecture	v6 migrated to skills with SKILL.md entypoints	Skills with SKILL.md + YAML frontmatter from start	Both have
Modular expansion packs	5 modules: BMM, BMB, TEA, BMGD, CIS	Single bundle, no module system	CTS gap
Scale-adaptive intelligence	Auto-adjusts depth (bug fix vs enterprise)	Manual via YAGNI + user picks complexity	CTS gap
Zero-dependency setup	Requires Node.js v20+, npm	Plain markdown, no build tools	BMAD gap
Total specialists	12+ agents (PM, Architect, Dev, SM, UX, QA, etc.)	19 specialists (PM, Scrum, Tech Lead, Security, DBA, UX, UI, Backend, Frontend, Tester, Test Architect, etc.)	BMAD gap
Dedicated Security Engineer	No dedicated agent; via DevOps expansion	Full: STRIDE, threat models, compliance, auth design	BMAD gap
Dedicated DBA	Database handled by Architect/Dev	Full: schema, queries, migrations, indexes, backup	BMAD gap
UI/UX split	Single UX Designer agent	Separate UX (flows/usability) + UI (visual/tokens)	BMAD gap
Marketing team	No marketing agents	3: Digital Marketer, Copywriter, Content Marketer	BMAD gap
Agent personas (names)	Named: Sarah Chen (PM), Winston (Architect), Amelia (Dev)	Role-based, no named personas	CTS gap
Agent customization	customize.yaml: override persona, memories, workflows	Edit SKILL.md manually, no override layer	CTS gap
Party Mode	Multiple agents debate in one session	Sequential: one specialist at a time via handoffs	CTS gap
Structured workflow phases	4 phases: Analysis → Planning → Solutioning → Implementation	6 phases: Understand → Brainstorm → Plan → Execute → Verify → Ship	Both have
Pre-built workflows	34+ workflows	13 workflow templates	Both have
Step-file architecture	Just-in-time step loading; only current step in context	Full SKILL.md loaded per specialist	CTS gap

Document-first chain	Strict: PRD → Architecture → Epics → Stories	★ FIXED: Full PRD → Architecture → Stories chain with templates + validation	★ FIXED
Blocker protocol	No explicit blocker handling	Structured: ■ BLOCKER with A/B/C options, logged	BMAD gap
Cross-specialist handoffs	Artifact-based (PRD feeds Architecture feeds Stories)	Handoff protocol: 3-line summary + inter-skill logging	Both have
bmad-help assistant	Built-in: 'what do I do next?'	QUICKSTART.md cheat sheet, no interactive help	CTS gap
Lazy skill loading	Step-file: only current step in context	One specialist at a time, references on demand	Both have
Token budget guide	General guidance (74-90% savings claimed)	Explicit targets per session type + anti-patterns	BMAD gap
YAGNI enforcement	Scale-adaptive (implicit)	Explicit YAGNI gate in every specialist + orchestrator	Both have
Activity logging	No built-in logging; relies on artifacts	8 structured log files + skill change tracking	BMAD gap
KPI measurement	No metrics tracking	10 KPIs: velocity, completion, bug rate, scope creep, etc.	BMAD gap
Session resumption	Context via artifacts, no session log	SESSION_END snapshots, auto-resume from checkpoint	BMAD gap
Decision log	Decisions in PRD/architecture artifacts	Dedicated decisions.md with timestamped entries	BMAD gap
Risk tracking	No dedicated risk log	risks.md with probability, mitigation, status	BMAD gap
Inter-skill + skill change log	No communication tracking	INTER_SKILL, HANDOFF, SKILL_CHANGE entries	BMAD gap
Report generation	No built-in reporting	Python script + on-demand status reports, retros	BMAD gap
Test Engineering Architect	TEA module: risk-based strategy, ATDD, automation	★ FIXED: Test Architect with risk scoring, ATDD/Gherkin, traceability matrix, release gates	★ FIXED
Adversarial review	Edge case hunter + adversarial review workflows	★ FIXED: Full adversarial checklist (input/auth/race/logic/data) + edge case hunter	★ FIXED
Security scanning refs	DevOps expansion pack	5 deep-dive references: code, CI/CD, containers, cloud, compliance	BMAD gap
OWASP Top 10	Not included	Full coverage with detection + remediation per vuln	BMAD gap
Compliance frameworks	Not included	SOC2, PCI-DSS, HIPAA, ISO 27001, NIST, GDPR	BMAD gap
Agent/skill builder	BMad Builder module for custom agents	INSTALL.md instructions for adding skills manually	CTS gap
Creative Intelligence Suite	CIS: brainstorming, storytelling, design thinking	★ FIXED: 7 frameworks (brainstorm, SCAMPER, design thinking, storytelling, first principles, reframing, naming)	★ FIXED
Game dev module	BMGD: Unity, Unreal, Godot workflows	Not included	CTS gap
Community & ecosystem	37K+ GitHub stars, MIT, active community	Private/personal project	CTS gap
Web bundle (ChatGPT/Gemini)	Compiled .txt bundles for web LLMs	Claude Code only	CTS gap

Gaps Fixed Since Initial Comparison

Gap	What Was Added
Creative Intelligence Suite	7 frameworks: brainstorming, SCAMPER, design thinking (5 phases), storytelling (Hero's Journey), first principles, problem reframing, name generation. Idea scoring matrix.
Document-First Chain	PRD → Architecture → Stories with full templates, validation checklists, traceability. YAGNI-scaled: full chain for large, mini-PRD for medium, skip for small.
Test Engineering Architect	Risk-based test strategy with scoring, ATDD/Gherkin specifications, coverage targets by risk level, traceability matrix, release gate criteria, NFR testing.
Adversarial Review	Full checklist: input abuse (SQLi, XSS, traversal), auth bypass (IDOR, CSRF, escalation), race conditions, business logic, data integrity. Edge case hunter. Finding format.
Skill Change Monitoring	Project Monitor now tracks SKILL_CHANGE events — when skills are added, modified, or removed. Logged to activity.md with affected files and routing updates.

Remaining CTS Gaps (Intentional Trade-offs)

These 8 BMAD features are intentionally skipped. CTS prioritizes simplicity, token efficiency, and zero-dependency setup over platform breadth and ecosystem size.

BMAD Feature	Why CTS Skips It
NPM installer	CTS is zero-dependency — copy markdown files, no Node.js needed.
Multi-IDE support (10+)	CTS targets Claude Code specifically for deeper, optimized integration.
Agent personas (named characters)	CTS uses role-based skills — simpler to maintain, fewer tokens per activation.
Party Mode (multi-agent debate)	CTS uses sequential handoffs with 3-line summaries — ~70% less token usage.
Modular expansion packs	CTS is one bundle — everything included, nothing to install separately.
Scale-adaptive intelligence	CTS uses explicit YAGNI gates — user picks complexity level, not auto-detected.
Agent-as-Code (YAML compilation)	CTS uses plain markdown — editable in any text editor, no build step.
Web bundles (ChatGPT/Gemini)	CTS is Claude Code native — one platform done well vs many done partially.